ИТМО

SPA-архитектура Frontend-приложения

Single Page Application

ИТМО

Вспоминаем...

AJAX (Asynchronous JavaScript and XML) - подход, при котором JavaScript обменивается данными с сервером и обновляет часть интерфейса без полной перезагрузки.

Примеры: поиск, фильтры, пагинация, формы, уведомления.

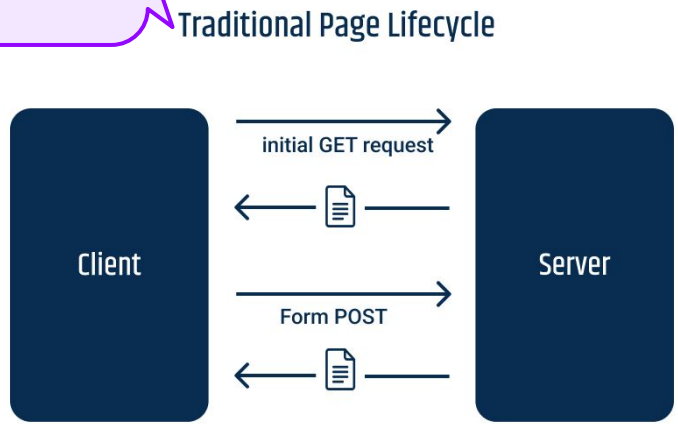
SPA

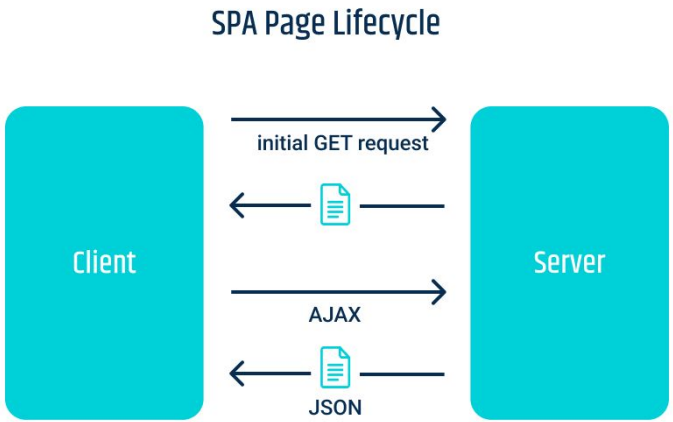
Single Page Application - архитектура/приложение, в рамках которого происходит загрузка и обновление единственного HTML-документа.

Все изменения (отправка форм, переходы, подгрузки данных) происходят динамически, с использование AJAX.

Single Page Application

MPA (multi-page application)





SPA компонент

ИТМО

Компонент - независимый переиспользуемый блок интерфейса, из которых состоит SPA.

Обычно является совокупностью шаблона (HTML-документ), локальных стилей (CSS) и класса, обслуживающего шаблон (JS/TS)

Пример Angular-компонента

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  title = 'example';
}
```

Справедливо не для всех!

Общие слои SPA

ИТМО

Router

Компоненты, отвечающие за маршрутизацию (связывают маршруты с конкретными компонентами)

Pages

Компоненты верхнего уровня, связанные с маршрутами. Получают параметры маршрута, агрегируют feature-компоненты

Features

Компонент раздел/пользовательской операции (форма ввода, таблица студентов или другие формы отображения данных)

Domain model

Предметные сущности, внутренние типы/классы/интерфейсы

API client

Сервисы, выполняющие запросы и базово обрабатывающие ответы

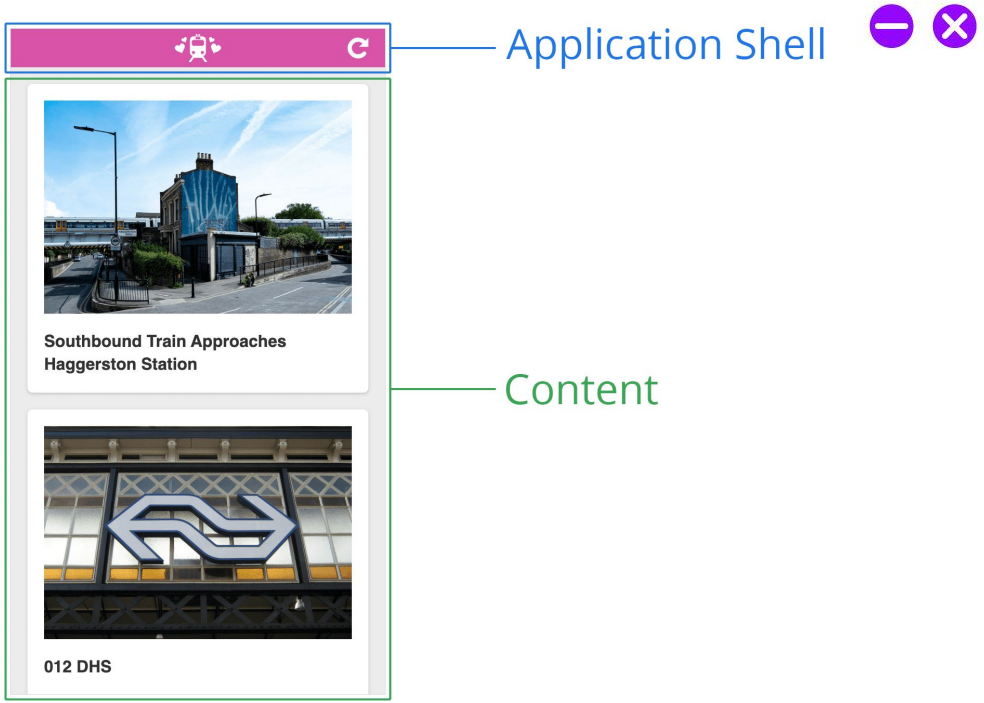
Shared

Общие и переиспользуемые для всех остальных компонентов элементы (в том числе Domain models)

App Shell

App Shell - архитектурный шаблон, отделяющий постоянные неизменяемые компоненты от динамического контента.

- Кэширование части файлов
- Быстрый рендеринг скелета
- Экономия трафика



App Shell

ИТМО

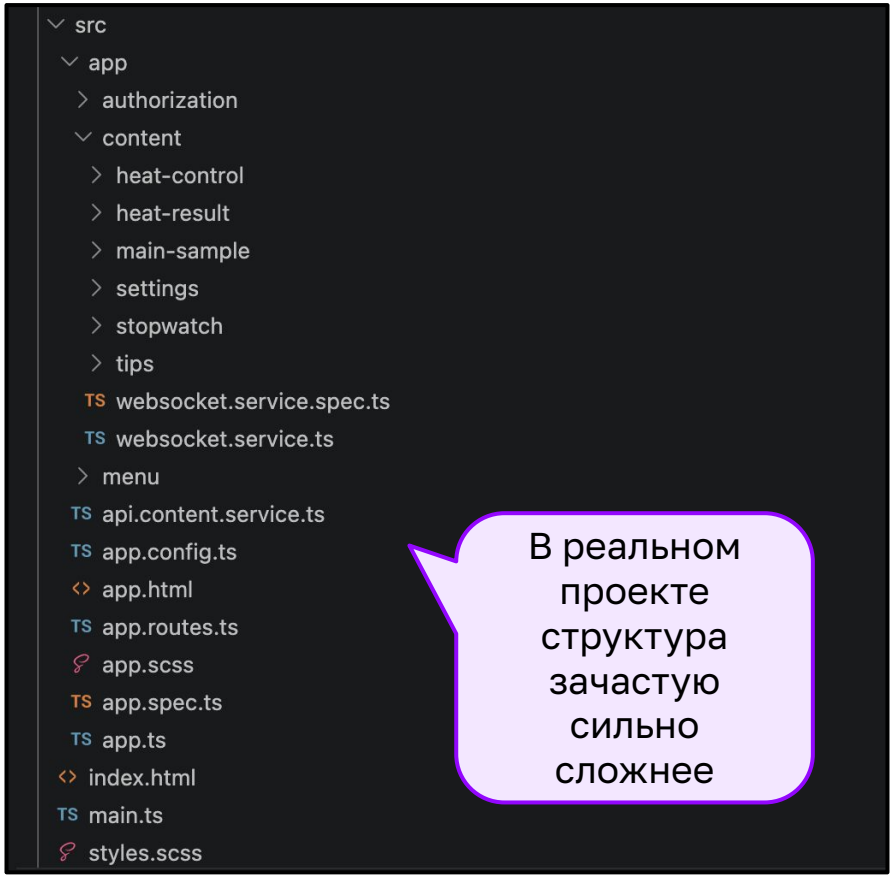
```
src/
├── index.ts                    # Main entry point
├── globals.css                 # Global styles
├── utils.ts                    # Shared utilities
├── initia-app-menu/            # App menu component
│   ├── index.tsx
│   ├── styles.css
│   ├── constants.ts
│   ├── types.ts
│   └── utils.ts
├── initia-check-icon/          # Check icon component
│   ├── index.tsx
│   └── styles.css
└── initia-terms/               # Terms modal component
    ├── index.tsx
    ├── styles.css
    ├── constants.ts
    └── utils.ts
```

Реализация React App Shell из сторонней библиотеки

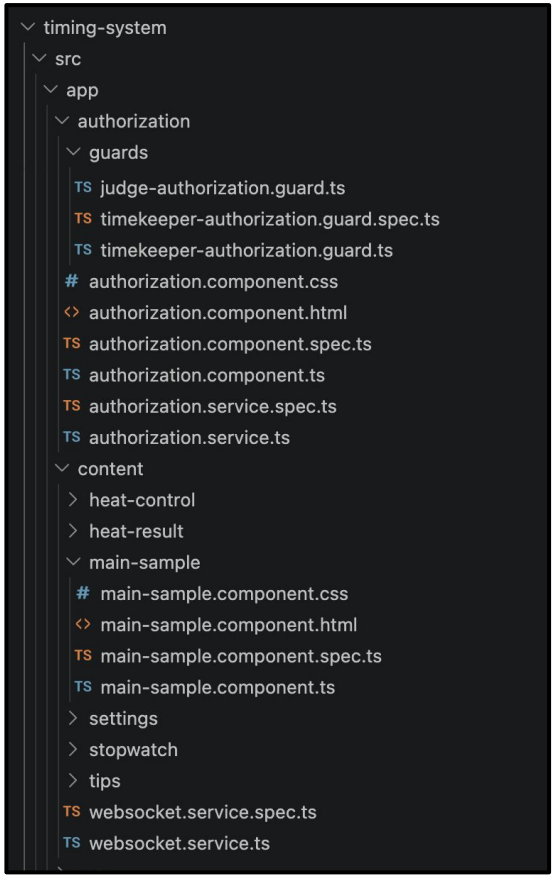
Вариант возможной реализации, не аксиомы

Пример SPA архитектуры

ИТМО



В реальном проекте структура зачастую сильно сложнее



Подсказка по слоям SPA

ИТМО

Router

Компоненты, отвечающие за маршрутизацию (связывают маршруты с конкретными компонентами)

Pages

Компоненты верхнего уровня, связанные с маршрутами. Получают параметры маршрута, агрегируют feature-компоненты

Features

Компонент раздел/пользовательской операции (форма ввода, таблица студентов или другие формы отображения данных)

Domain model

Предметные сущности, внутренние типы/классы/интерфейсы

API client

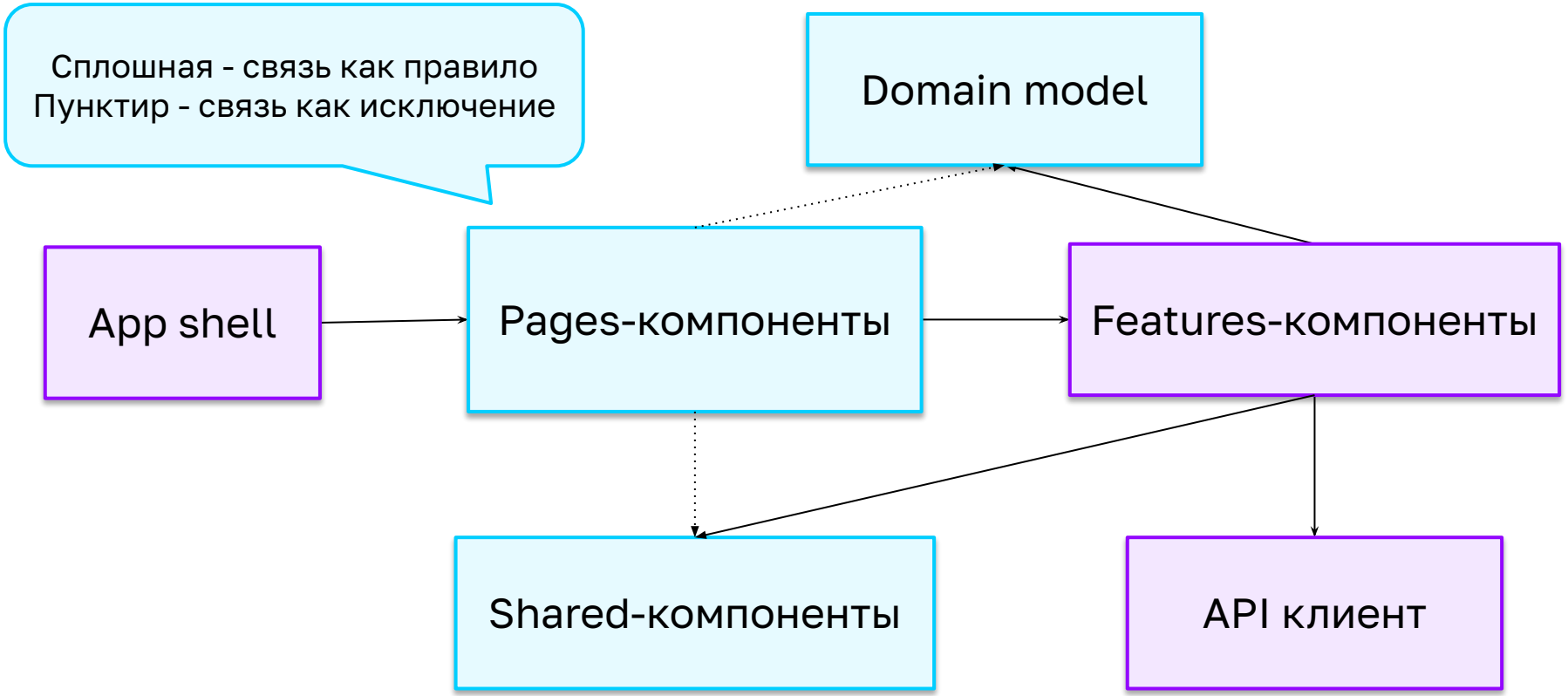
Сервисы, выполняющие запросы и базово обрабатывающие ответы

Shared

Общие и переиспользуемые для всех остальных компонентов элементы (в том числе Domain models)

Направление зависимостей

ИТМО



Маршрутизация

ИТМО

```
/catalog  → компонент каталога
/catalog/42  → объект с идентификатором 42
/catalog?sort=price → каталог с сортировкой
```

- Связываем определенный URL с состоянием интерфейса
- Обычно представлены объектами Router/схожее название
- Передача параметров в конкретный feature-компонент, требование аутентификации и тп

Маршрутизация на примере Vue js

ИТМО

```vue
<script setup>
import { ref, computed } from 'vue'
import Home from './Home.vue'
import About from './About.vue'
import NotFound from './NotFound.vue'

const routes = {
  '/': Home,
  '/about': About
}

const currentPath = ref(window.location.hash)

window.addEventListener('hashchange', () => {
  currentPath.value = window.location.hash
})

const currentView = computed(() => {
  return routes[currentPath.value.slice(1) || '/'] || NotFound
})
</script>

<template>
  <a href="#/">Home</a> |
  <a href="#/about">About</a> |
  <a href="#/non-existent-path">Broken Link</a>
  <component :is="currentView" />
</template>
```

Маршрутизация на примере Angular

ИТМО

```
export const routes: Routes = [
  { path: 'auth', component: AuthorizationComponent },
  {
    path: 'judge',
    component: MainSampleComponent,
    canActivate: [JudgeAuthorizationGuard],
    canActivateChild: [JudgeAuthorizationGuard],
    children: [
      { path: 'scoreboard', component: HeatResultComponent },
      { path: 'settings', component: SettingsComponent },
      { path: 'control', component: HeatControlComponent },
      { path: 'tips', component: TipsComponent },
      { path: '**', redirectTo: 'ssettings' },
    ],
  },
  {
    path: 'timekeeper',
    component: StopwatchComponent,
    canActivate: [JudgeAuthorizationGuard],
    canActivateChild: [JudgeAuthorizationGuard],
  },
  { path: '**', redirectTo: 'auth' },
];
```

Можно настроить передачу query-параметров

Клиентская навигация

ИТМО

Hash routing

Техника клиентской навигации, при которой состояние интерфейса зависит от части пути, расположенного после #

Хранится в window.location.hash, переключение по событию hashchange

History API

Браузерный интерфейс (window.history), предоставляющий возможность управлять историей сессии и URL без перезагрузки страницы

Несколько методов: pushState, replaceState, back и тд

Пример Hash routing

ИТМО

```
<body>
  <nav>
    <a href="#/">Главная</a>
    <a href="#/about">О нас</a>
  </nav>

  <main id="app"></main>

  <script>
    const pages = {
      "/": "<h1>Главная</h1><p>Добро пожаловать!</p>",
      "/about": "<h1>О нас</h1><p>Мы изучаем роутинг.</p>"
    };

    function render() {
      // Например: "#/about" → "/about"
      const path = location.hash.slice(1) || "/";

      document.getElementById("app").innerHTML =
        pages[path] ?? "<h1>Страница не найдена</h1>";
    }

    window.addEventListener("hashchange", render);

    render(); // Показываем страницу при первом открытии
  </script>
</body>
```

Пример Hash routing

ИТМО

```html
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <title>Hash routing</title>
</head>
<body>
  <nav>
    <a href="#/">Главная</a>
    <a href="#/about">О нас</a>
  </nav>

  <main id="app"></main>

  <script>
    const pages = {
      "/": "<h1>Главная</h1><p>Добро пожаловать!</p>",
      "/about": "<h1>О нас</h1><p>Мы изучаем роутинг.</p>"
    };

    function render() {
      // Например: "#/about" → "/about"
      const path = location.hash.slice(1) || "/";

      document.getElementById("app").innerHTML =
        pages[path] ?? "<h1>Страница не найдена</h1>";
    }

    window.addEventListener("hashchange", render);

    render(); // Показываем страницу при первом открытии
  </script>
</body>
</html>
```

Главная О нас

Главная

Добро пожаловать!

```html
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <title>Hash routing</title>
</head>
<body>
  <nav>
    <a href="#/">Главная</a>
    <a href="#/about">О нас</a>
  </nav>

  <main id="app"></main>

  <script>
    const pages = {
      "/": "<h1>Главная</h1><p>Добро пожаловать!</p>",
      "/about": "<h1>О нас</h1><p>Мы изучаем роутинг.</p>"
    };

    function render() {
      // Например: "#/about" → "/about"
      const path = location.hash.slice(1) || "/";

      document.getElementById("app").innerHTML =
        pages[path] ?? "<h1>Страница не найдена</h1>";
    }

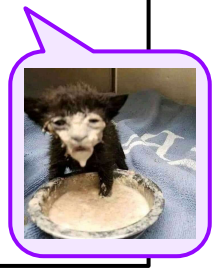
    window.addEventListener("hashchange", render);

    render(); // Показываем страницу при первом открытии
  </script>
</body>
</html>
```

Главная О нас

О нас

Мы изучаем роутинг.



Пример History API

ИТМО

```html
<body>
  <nav>
    <a href="/">Главная</a>
    <a href="/about">О нас</a>
  </nav>

  <main id="app"></main>

  <script>
    const pages = {
      "/": "<h1>Главная</h1><p>Добро пожаловать!</p>",
      "/about": "<h1>О нас</h1><p>Мы изучаем роутинг.</p>"
    };

    function render() {
      const path = location.pathname;

      document.getElementById("app").innerHTML =
        pages[path] ?? "<h1>Страница не найдена</h1>";
    }
```

```
    document.querySelectorAll("nav a").forEach(link => {
      link.addEventListener("click", event => {
        event.preventDefault(); // Отменяем загрузку страницы

        history.pushState(null, "", link.getAttribute("href"));

        render();
      });
    });

    window.addEventListener("popstate", render);

    render();
  </script>
</body>
```

Пример History API

ИТМО

```html
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <title>History API routing</title>
</head>
<body>
  <nav>
    <a href="/">Главная</a>
    <a href="/about">О нас</a>
  </nav>

  <main id="app"></main>

  <script>
    const pages = {
      "/": "<h1>Главная</h1><p>Добро пожаловать!</p>",
      "/about": "<h1>О нас</h1><p>Мы изучаем роутинг.</p>"
    };

    function render() {
      const path = location.pathname;

      document.getElementById("app").innerHTML =
        pages[path] ?? "<h1>Страница не найдена</h1>";
    }

    document.querySelectorAll("nav a").forEach(link => {
      link.addEventListener("click", event => {
        event.preventDefault(); // Отменяем загрузку страницы

        history.pushState(null, "", link.getAttribute("href"));

        render(); // pushState сам не вызывает popstate
      });
    });

    // Реакция на кнопки браузера «Назад» и «Вперёд»
    window.addEventListener("popstate", render);

    render();
  </script>
</body>
</html>
```

Главная О нас

Главная

Добро пожаловать!

Console


```html
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <title>History API routing</title>
</head>
<body>
  <nav>
    <a href="/">Главная</a>
    <a href="/about">О нас</a>
  </nav>

  <main id="app"></main>

  <script>
    const pages = {
      "/": "<h1>Главная</h1><p>Добро пожаловать!</p>",
      "/about": "<h1>О нас</h1><p>Мы изучаем роутинг.</p>"
    };

    function render() {
      const path = location.pathname;

      document.getElementById("app").innerHTML =
        pages[path] ?? "<h1>Страница не найдена</h1>";
    }

    document.querySelectorAll("nav a").forEach(link => {
      link.addEventLister("click", event => {
        event.preventDefault(); // Отменяем загрузку страницы

        history.pushState(null, "", link.getAttribute("href"));

        render(); // pushState сам не вызывает popstate
      });
    });

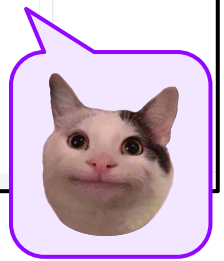
    // Реакция на кнопки браузера «Назад» и «Вперёд»
    window.addEventListener("popstate", render);

    render();
  </script>
</body>
</html>
```


Главная О нас

О нас

Мы изучаем роутинг.



Navigation API

ИТМО

Navigation API - современный интерфейс клиентской навигации, пришедший на смену History API

- window.navigation
- Переходы/навигация между состояниями - событие navigate
- Асинхронный handler() для ожидания загрузки

```
<script>
  const pages = {
    "/": "<h1>Главная</h1><p>Добро пожаловать!</p>",
    "/about": "<h1>О нас</h1><p>Мы изучаем роутинг.</p>"
  };

  function render(path) {
    document.getElementById("app").innerHTML =
      pages[path] ?? "<h1>Страница не найдена</h1>";
  }

  window.navigation.addEventListener("navigate", event => {
    // Внешние переходы, якоря и скачивания
    // оставляем браузеру
    if (
      !event.canIntercept ||
      event.hashChange ||
      event.downloadRequest !== null
    ) {
      return;
    }
```

Пример Navigation API

ИТМО

```
    const url = new URL(event.destination.url);

    // Обрабатываем переход без загрузки нового документа
    event.intercept({
      async handler() {
        render(url.pathname);
      }
    });
  });

  // При первом открытии страницу рисуем самостоятельно
  render(location.pathname);
</script>
```



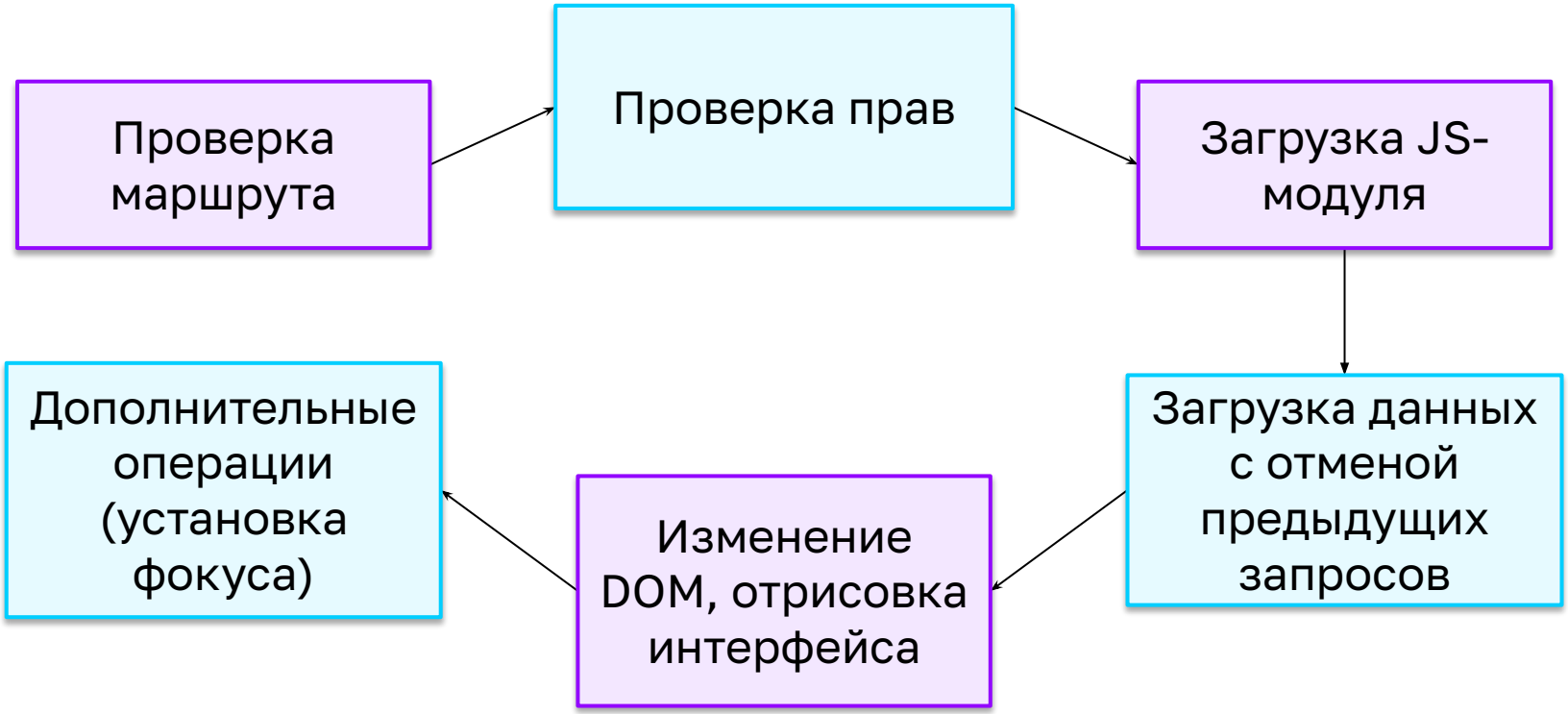
Сравнительная таблица

ИТМО

	Hash routing	History API	Navigation API
Адрес страницы	`/#/about`	`/about`	`/about`
Переход по ссылке	Изменение hash	Перехват события + pushState()	navigate
Назад/вперед	hashchange	popstate	navigate
Ожидание данных	Сами	Сами	handler()
Отмена перехода	Сами	Сами	event.signal

Общий Navigation lifecycle

ИТМО



State management

ИТМО

Состояние - минимальный набор данных, определяющих текущее состояние UI

Виды:

- UI - про элементы интерфейса
- From - про формы
- URL - параметры представления в URL
- Derived - полученные из других данных
- и другие...



```
// Исходное состояние
const courses = [
  { title: "JavaScript", completed: true },
  { title: "Angular", completed: false }
];

const showOnlyCompleted = true;

// Вычисляемое состояние
const visibleCourses = courses.filter(course =>
  showOnlyCompleted ? course.completed : true
);
```

Состояния по времени жизни

ИТМО

```
let searchQuery = "";
searchQuery =
"JavaScript";
```

Временные

- Существуют в рамках компонента
- Пример - состояние кнопки в форме

```
sessionStorage.setItem("dr
aft", "Мой ответ");
const draft =
sessionStorage.getItem("dr
aft") ?? "";
```

Сессионное

- Существует в рамках сессии
- Пример - access-токен

```
localStorage.setItem("them
e", "dark");
const theme =
localStorage.getItem("them
e") ?? "light";
```

Долговременные

- Сохраняется между открытиями приложения
- Пример - пользовательские темы

Состояния по видимости

ИТМО

```
function Card() {
  let isExpanded = false;
}
```

Локальные

- Используются только внутри компонента
- Пример - то же состояние кнопки

```
const filters = {
completed: true };
reFilterPanel(filters);
reCourseList(filters);
```

Общие/разделенные

- Используются в рамках нескольких компонентов
- Пример - выбранные фильтры формы

```
// app-state.js —
компоненты импортируют это
состояние

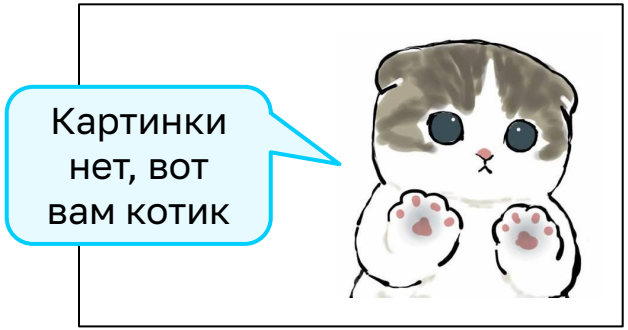
export const appState = {
language: "ru" };
```

Глобальные

- Используются множеством компонентов в рамках всего приложения
- Пример - выбранный язык

Паттерны работы со state

ИТМО



Поднятие состояния

```
const selUser = users.find(
   us => us.id ===
selUserId);

const delUser = users.find(
   us => us.id ===
delUserId);
```

Единый источник правды

```
const updatedUser = {
   ...user,
   name: "Alex"
 };
```

Иммутабельность

```
function selActUsers(state:
State): User[] {
   return state.userIds
      .map(id =>
state.users[id])
      .filter(user =>
user.active);
 }
```

Селекторы

```
type AsyncState<T> =
   | { status: "idle" }
   | { status: "loading" }
   | { status: "success";
data: T }
   | { status: "error";
error: string };
```

Конечный автомат

```
newState =
reducer(previousState,
action)

type Action =
   | { type: "loaded"; data:
User[] }
   | { type: "failed";
error: string };
```

Reducer

Формы в SPA

ИТМО

```jsx
import { useState } from "react";

function ControlledForm() {
  const [name, setName] = useState("");

  function handleSubmit(event) {
    event.preventDefault();
    console.log(name);
  }

  return (
    <form onSubmit={handleSubmit}>
      <input aria-label="Имя" value={name}
        onChange={event => setName(event.target.value)} />
      <button>Отправить</button>
    </form>
  );
}
```

Пример на React

- Формы в SPA остаются теми же элементами с той же конструкцией
- Делятся на controlled и uncontrolled
- Controlled - состояние формы хранится в компоненте

Constraint Validation API

ИТМО

```
<input
  aria-label="Возраст"
  type="number"
  min="18"
  required
  onBlur={event => {
    console.log(event.target.checkValidity());
    console.log("Возраст меньше 18:");
    console.log(event.target.validity.rangeUnderflow);
  }}
/>
```

```
function EmailForm() {
  function handleSubmit(event) {
    event.preventDefault();
    if (!event.currentTarget.reportValidity()) return;
    console.log("Форма заполнена корректно");
  }

  return (
    <form noValidate onSubmit={handleSubmit}>
      <input aria-label="Email" type="email" required />
      <button>Отправить</button>
    </form>
  );
}
```

CV API - встроенный JS-интерфейс для проверки корректности введенных в HTML-форме данных.

- checkValidity()
- reportValidity()
- validity
- validationMessage
- setCustomValidity()

Валидация

ИТМО

Асинхронная валидация

Может требовать ожидания некоторой операции, например:

- Занято ли имя
- Существует ли сущность
- Разрешена ли операция

Накладывают свои ограничения:

- Отмена предыдущего запроса
- Состояние у проверки
- Race condition

Синхронная валидация



Стандартная клиентская валидация

Ну правда

- Проверка на длину
- Проверка на формат
- Обязательность

API client

ИТМО

```
@Injectable({
  providedIn: 'root'
})
export class ApiContentService {

  constructor(private http: HttpClient) {}

  baseUrl = 'http://localhost:3000/api';

  createHeader() {
    let headers = new HttpHeaders({
      'Content-Type': 'application/json'
    });

    const access = localStorage.getItem('access');
    if (access) {
      headers = headers.set('Authorization', `Basic ${window.btoa(access)}`);
    }

    return { headers };
  }

  createUser(UserData: any) {
    const url = `${this.baseUrl}/create_user/`;
    return this.http.post(url, UserData, this.createHeader());
  }
```

API client на примере Angular

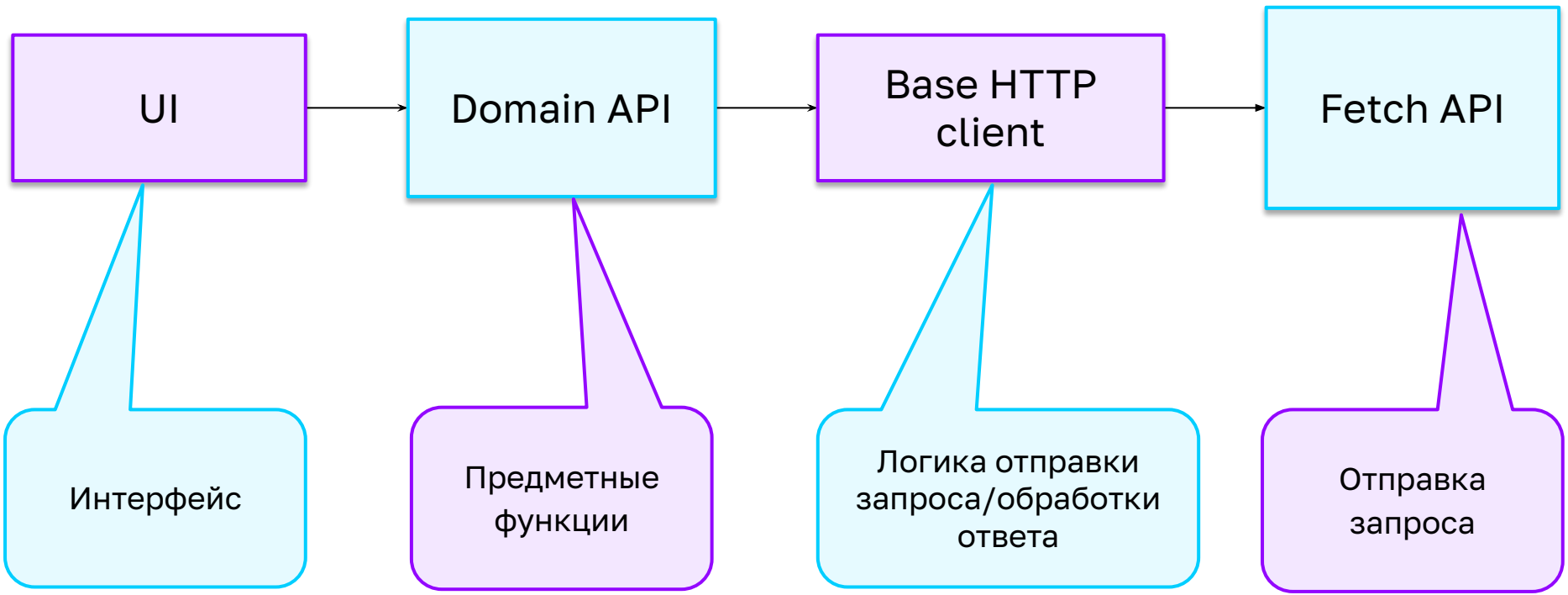
API client - подход к проектированию интерфейса взаимодействия клиента с сервером в SPA

Что дает?

- Изоляция состояния
- Единый источник правды
- Упрощает рефакторинг

API client

ИТМО



API client

ИТМО

Domain API

Предметные функции запроса данных

Изолируют UI от деталей реализации HTTP клиента

```
createUser(UserData: any) {
  const url = `${this.baseUrl}/create_user/`;
  return this.http.post(url, UserData, this.createHeader());
}
```

Base HTTP client

Базовый HTTP-client

Общие технические детали запроса (метод, адрес, чтение ответа)

```
const httpClient = {
  async get(path) {
    const response = await fetch(`/api${path}`);

    if (!response.ok) {
      throw new Error(`HTTP ${response.status}`);
    }

    return response.json();
  }
};
```

API client

ИТМО

Domain API

Предметные функции запроса данных

Изолируют UI от деталей HTTP клиента

```
createUser(UserData: any) {
  const url = `${this.baseUrl}/create_user/`;
  return this.http.post(url, UserData, this.createHeader());
}
```

Нарушение!
Не делайте так

Base HTTP client

Базовый HTTP-client

Общие технические детали запроса (метод, адрес, чтение ответа)

```
const httpClient = {
  async get(path) {
    const response = await fetch(`/api${path}`);

    if (!response.ok) {
      throw new Error(`HTTP ${response.status}`);
    }

    return response.json();
  }
};
```

И еще кое-что про API client

ИТМО

- Используем DTO и мапперы (аналогично серверной стороне)
- Делаем свою модель ошибок и реагируем в зависимости от нее (аналогично серверной стороне)
- Отменяем запросы, если они не актуальны
- Выбираем optimistic/pessimistic мутации по ситуации
  - Optimistic - сначала изменение UI, потом запрос
  - Pessimistic - наоборот
- Кэшируем при возможности



Тестирование

ИТМО

Тестирование - процесс проверки соответствия реального и ожидаемого поведения системы

- Unit
- Component
- Integration
- E2E

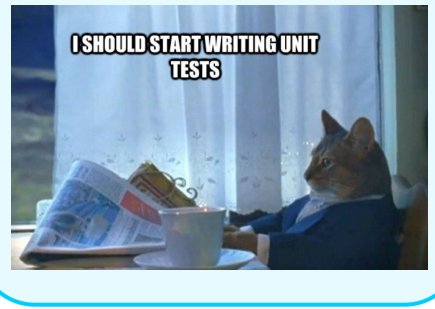
Да, оно реально нужно

```
describe('MainSampleComponent', () => {
  let component: MainSampleComponent;
  let fixture: ComponentFixture<MainSampleComponent>;

  beforeEach(async () => {
    await TestBed.configureTestingModule({
      declarations: [ MainSampleComponent ]
    })
    .compileComponents();

    fixture = TestBed.createComponent(MainSampleComponent);
    component = fixture.componentInstance;
    fixture.detectChanges();
  });

  it('should create', () => {
    expect(component).toBeTruthy();
  });
});
```



Vitest

ИТМО

Что это

Vitest - фреймворк модульного и компонентного тестирования на JS и TS. Конфигурация происходит в vitest.config.js через defineConfig

Предоставляет: describe и it, assertions, mocks/spies, coverage, snapshot и многое другое

Как это

```
describe("calculateTotal", () => {
   it("вычисляет сумму", () => {
      expect(
      calculateTotal(100, 2)
      ).toBe(200);
   });
});
```

Vitest

ИТМО

Arrange - подготовка данных

```
describe("calculateTotal", () => {
  it("вычисляет сумму", () => {
    const expectedTotal = 200;

    const total =
calculateTotal(100, 2);

expect(total).toBe(expectedTotal);
  });
});
```

Act - действие

Assert - проверка

jsdom/happy-dom

ИТМО

```
// @vitest-environment jsdom
import { test, expect } from "vitest";

test("кнопка меняет текст после нажатия", () => {
  document.body.innerHTML = "<button>Нажми меня</button>";
  const button = document.querySelector("button");

  button.addEventListener("click", () => {
    button.textContent = "Нажато!";
  });

  button.click();

  expect(button.textContent).toBe("Нажато!");
});
```

Реализации DOM на JS без графического интерфейса (headless)

- Быстрее, чем использовать настоящий браузер
- Поведение может отличаться

Vitest + jsdom

ИТМО

```
// vitest.config.js
import { defineConfig } from "vitest/config";

export default defineConfig({
  test: {
    environment: "jsdom"
  }
});

// button.test.js
import { test, expect, beforeEach, afterEach } from "vitest";

beforeEach(() => {
  document.body.innerHTML = "<button>Нажми меня</button>";

  const button = document.querySelector("button");

  button.addEventListener("click", () => {
    button.textContent = "Нажато!";
  });
});

afterEach(() => {
  document.body.replaceChildren();
});

test("кнопка показывает начальный текст", () => {
  const button = document.querySelector("button");

  expect(button.textContent).toBe("Нажми меня");
});
```

Конфигурация jsdom

```
// vitest.config.js
import { defineConfig } from "vitest/config";
import { playwright } from "@vitest/browser-playwright";

export default defineConfig({
  test: {
    browser: {
      enabled: true,
      provider: playwright(),
      instances: [{ browser: "chromium" }],
      headless: true
    }
  }
});
```

Конфигурация реального браузера (через Playwright)

Тестирование

ИТМО

Mock

Объект, подменяющий зависимости с необходимым заданным поведением/свойствами.

```
import { vi, expect } from "vitest";

const api = {
  getUser: vi.fn().mockResolvedValue({ name: "Анна" })
};

const user = await api.getUser(42);

expect(user.name).toBe("Анна");
expect(api.getUser).toHaveBeenCalledWith(42);
```

Spy

Объект-обертка над реальной функцией, записывающая вызовы. По-умолчанию не изменяет свойства/поведение (но может)

```
import { vi, expect } from "vitest";

const spy = vi.spyOn(console, "log");

console.log("Привет"); // Настоящий console.log выполняется

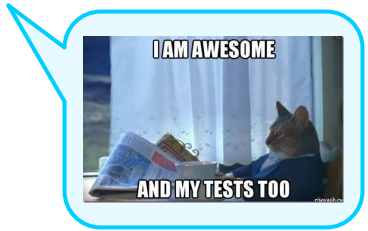
expect(spy).toHaveBeenCalledWith("Привет");
expect(spy).toHaveBeenCalledTimes(1);

spy.mockRestore(); // Восстанавливаем исходный метод
```

И еще кое-что про тестирование

ИТМО

- Интерфейс тестируем с точки зрения пользователя (имитируем его поведение)
- Для поиска элемента на интерфейсе используем семантические запросы (поиск по label, а не стилю или сложному селектору)
- Изолируем окружение тестируемого элемента по максимуму (мокаем все зависимости)
- Используем в тесте тестовые данные (реальные данные могут меняются)



Спасибо
за внимание!

ITMO more than a UNIVERSITY